
Rapport - Projet Karaoke

Date	09/11/2024
Membres	TECHER Noah HOAREAU Anthony ILATA Miguel

Table des matières

I. Introduction.....	3
A. Présentation du projet.....	3
B. Topologie.....	4
C. Répartition des tâches.....	4
II. Notice d'utilisation.....	5
A. Fonctionnalités.....	5
B. Prérequis.....	5
C. Lancement.....	5
D. Utilisation.....	6
III. Technologies utilisées.....	6
A. APIs.....	6
B. Langage de programmation.....	7
C. Base de données.....	7
IV. Axe d'amélioration.....	8
A. File d'attente.....	8
B. Changer d'API.....	8
C. Mise à jour du formulaire.....	9
D. Défilement des paroles.....	9
V. Annexe.....	9
A. app.py.....	9
B. index.html.....	10
C. index_finish.html.....	10
D. karaoke.html.....	11
E. 404.html.....	11

I. Introduction

A. Présentation du projet

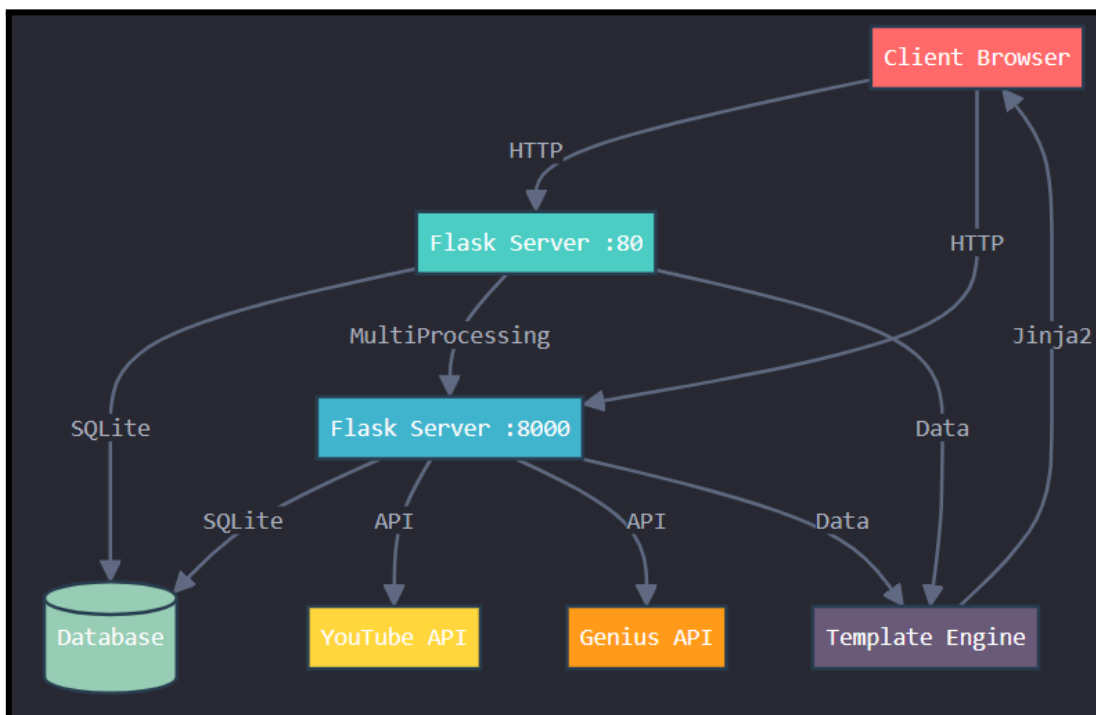
Pour la SAE 302 axée sur le développement d'applications communicantes, notre équipe a choisi de réaliser une application web de karaoké. Ce projet met en œuvre différents aspects de la communication entre systèmes informatiques tout en offrant une expérience utilisateur ludique et interactive.

Notre application permet aux utilisateurs de soumettre leurs chansons (3 langues possible pour l'instant anglais, français et espagnol) via une page dédiée et de chanter leurs morceaux préférés avec les paroles affichées sur une autre page. Le système gère automatiquement une file d'attente grâce à une base de données, offrant ainsi une expérience de karaoké où chacun peut participer tour à tour.

Nous nous sommes inspirés des salles de karaoké qui existent. Ce sont de petites salles avec lesquelles on retrouve un grand écran avec dessus la vidéo et les paroles de la chanson et aussi une tablette avec laquelle on peut choisir sa chanson.

Ce projet est dans le thème d'une application communicante puisque que dans celle-ci il y a aura de la communication avec:

- Client/Serveur
- Différentes API (Génius et Youtube)
- Une base de données SQL
- ect



B. Topologie

```
karaoke/
├── app.py           # Application principale Flask
├── api.py           # Fonctions d'API pour YouTube et Genius
├── data/
│   └── chansons.db  # Base de données SQLite
├── templates/
│   ├── index.html   # Page du formulaire
│   ├── karaoke.html  # Page de diffusion
│   ├── index_finish.html # Page de confirmation
│   └── 404.html      # Page d'erreur 404 (ou erreur 500)
└── static/
    ├── css/         # Styles CSS
    │   ├── index.css
    │   ├── index_finish.css
    │   └── karaoke.css
```

C. Répartition des tâches

Pour ce projet nous étions une équipe de trois personnes: TECHER Noah, HOAREAU Anthony et ILATA Miguel. Nous avons réparti les tâches comme suit:

- TECHER Noah, s'est occupé de réaliser le serveur (*app.py*) qui permet de faire le lien entre les différents fichiers, il a géré la récupération des données grâce aux APIs (*api.py*) et enfin il a géré la gestion d'erreur (*404.html*).
- HOAREAU Anthony de son côté a effectué toute la partie client qui comporte la gestion des formulaires (*index.html* et *index_finish.html*) et la création de la base de données SQL (*chansons.db*).
- ILATA Miguel quant à lui a créé la page de lecture où l'on retrouve la vidéo et les paroles de la chanson (*karaoke.html*), de plus il a réalisé tout le style de l'application (*index.css*, *index_finish.css* et *karaoke.css*).
- Et tous ensemble nous avons réalisé ce rapport.

II. Notice d'utilisation

A. Fonctionnalités

Cette application web vous retrouver:

- Un formulaire pour soumettre des chansons avec nom d'utilisateur, titre et artiste de la chanson
- Lecture de vidéos YouTube intégrées
- Affichage automatique des paroles
- Système de file d'attente pour les chansons (BETA)
- Interface utilisateur intuitive et responsive

B. Prérequis

Pour utiliser cette application, vous aurez besoin des éléments suivants : une connexion Internet stable pour lire les vidéos YouTube, ainsi que Python 3.8 ou une version supérieure, avec certaines dépendances supplémentaires.

Pour installer les dépendances, ouvrez un terminal dans le dossier karaoke puis taper les commandes suivantes:

```
pip install flask
pip install youtube-search-python
pip install youtube-transcript-api
pip install lyricsgenius
```

C. Lancement

Une fois que vous avez tout les prérequis, vous pouvez enfin lancer l'application soit en double cliquant sur *app.py* soit en ouvrant un terminal dans le dossier en tapant:

```
python app.py
```

L'application lancera deux serveurs :

- Serveur du formulaire: *http://<votre adresse IP>:80*

Ce serveur sert à remplir un formulaire afin de choisir sa chanson.

De plus, il sera accessible sur tous les équipements de votre réseau en tapant dans un navigateur votre adresse IP.

- Serveur de lecture: *http://localhost:8000*

Ce sert sert à afficher les paroles et la vidéo

Il sera accessible uniquement sur l'ordinateur qui aura lancé le serveur pour des raison de droit.

D. Utilisation

Étape 1 - Accédez à la page de formulaire (port 80)

Étape 2 - Soumettez une chanson avec :

- Votre nom d'utilisateur
- Le nom de l'artiste
- Le titre de la chanson

Étape 3 - Accédez à la page de karaoké (port 8000) sur l'ordinateur qui l'a lancé

Étape 4 - Attendez votre tour

Étape 5 - Chanter !!!

III. Technologies utilisées

A. APIs

L'application utilise deux API externes

- YouTube Data API (via *youtube-search-python* et *youtube_transcript_api*)

Cette API permet nous permet de récupérer les vidéos et les transcriptions des différentes chansons sur Youtube.

- Genius API pour les paroles

Genius est une API qui contient les paroles des chansons, mais nous l'utilisons uniquement si les paroles ne se trouvent pas dans la transcription youtube. Puisque que Genius ne donne pas les time code à l'instar de la transcription youtube.

Pour son utilisation nous avons besoin d'un token:

"cBe8YGq_s7vMRuVRx4L0XAiam8_zRAH8aX1yJaSadFN0dqJ5rwPVfK6eOuhFXfeQ"

B. Langage de programmation

Pour réaliser ce karaoké, nous avons utilisé plusieurs langages de programmation comme Python, JavaScript, HTML/CSS (même si ce n'est pas un vrai langage de programmation).

Le Python est un langage de programmation backend qui permet de gérer la logique côté serveur, traiter les données et interagir avec la base de données. Nous avons utilisé Flask, un framework léger, pour développer l'application principale (*app.py*). Ce fichier est responsable du routage des requêtes, de la gestion des vues et de l'intégration avec la base de données SQLite pour stocker et récupérer les chansons. En complément, le fichier *api.py* contient des fonctions permettant d'interagir avec des APIs externes, comme celles de YouTube et Genius, afin d'obtenir les vidéos musicales et les paroles des chansons. Nous avons choisi Python au lieu de Php ou Java car, nous sommes d'ancien lycéen qui avait pris la spécialité NSI pour le bac. Nous avons donc de fortes compétences en Python.

Le JavaScript, quant à lui, est utilisé principalement pour l'interactivité du côté client. Il permet de rendre l'interface dynamique et de gérer des événements. Ce dernier est intégré dans les pages HTML, principalement dans *karaoke.html*, où les éléments interactifs sont contrôlés.

Les fichiers HTML et CSS forment l'aspect visuel et la structure du projet. Le fichier *index.html* contient le formulaire permettant à l'utilisateur de soumettre une chanson à jouer. La page *karaoke.html* est celle où la chanson et les paroles sont affichées pour l'utilisateur, tandis que *index_finish.html* affiche une page de confirmation. Le CSS, situé dans le dossier *static/css/*, est utilisé pour styliser ces pages et garantir une expérience visuelle agréable et fonctionnelle. Les fichiers *index.css*, *index_finish.css* et *karaoke.css* assurent la mise en forme de chaque page selon ses spécificités.

C. Base de données

Nous avons besoin d'une base de données qui pourrait contenir des informations telles que un nom d'utilisateur, le titre de la chanson avec le nom de l'artiste et enfin un ID pour comme clé primaire. Nous avons donc choisi une BDD SQL qu'on a configurée avec l'application SQLite qui nous a permis de mettre la base de données en tant que fichier (*chansons.db*). L'ID des chansons nous a permis de réaliser une file d'attente.

Nom	Type de données	Clé primaire
ID	INTEGER	
TITLE	TEXT	
ARTIST	TEXT	
PSEUDO	TEXT	

IV. Axe d'amélioration

A. File d'attente

Dans notre projet actuel on a déjà une file d'attente qui permet de faire défiler les chansons les une après les autres dans l'ordre où elles sont transmises. Mais cette file d'attente peut-être améliorée.

Premièrement, il faudrait changer l'ordre de priorité, imaginons qu'un utilisateur choisisse 10 chansons avant tout le monde alors c'est lui qui chantera pendant 10 chansons et c'est pour cela qu'on voudrait ajouter une nouvelle entrée dans la base de données qui se nommerait "nombre de réservation". Grâce à cette nouvelle entrée nous pourrions faire en sorte que même s'il y a des réservations avant plusieurs chansons, s'il a chanté plus de chansons que quelqu'un d'autre alors il la file d'attente le fera passer après ceux qui ont moins chanté.

Deuxièmement, il faudrait changer l'affichage du nombre de chansons qu'il y a avant la nôtre dans *index_finish.html*. Pour l'instant l'affichage est statique alors qu'avec du javascript on pourrait le mettre dynamique pour qu'il décroisse de 1 à chaque nouvelle chanson qui passe.

B. Changer d'API

Comme vous le savez on a choisi l'API Youtube pour récupérer la vidéo, mais avant ça on avait aussi essayé avec l'API de Spotify mais on a dû abandonner cette idée à cause de problèmes de token. On a eu un problème avec l'API Youtube celui-ci étant que on ne pouvait qu'afficher leur vidéo uniquement sur le localhost d'une machine dans le cas contraire la vidéo n'était plus disponible comme vous pouvez le voir ci-dessous :



Mais en réalité cela ne nous posait pas réellement problème car dans les karaoke il n'y a pas plusieurs écran mais bien un seul. Mais dans le futur le mieux ce serait de trouver une API qui permettent d'avoir plusieurs écrans pour la vidéo.

C. Mise à jour du formulaire

Quand on envoie le formulaire, l'application nous renvoie sur *index_finish.html* avec les informations qu'on vient de transmettre. Le problème étant si on veut choisir une nouvelle chanson on doit soit faire retour ou soit ouvrir un nouvel onglet, alors qu'il faudrait dans *index_finish.html* avoir un bouton qui nous ramène sur *index.html*. Et qu'en on enverra le second formulaire et qu'il nous amènera sur *index_finish.html* alors les informations des deux formulaires devraient s'y trouver.

D. Défilement des paroles

La dernière amélioration que nous souhaitons apporter à notre karaoké serait d'intégrer du JavaScript pour afficher les paroles de manière dynamique et synchronisée avec la musique. Nous avons déjà le time code, mais nous n'avons pas encore eu le temps de l'exploiter pour réaliser cette fonctionnalité. L'idée serait de faire défiler les paroles en temps réel afin qu'elles apparaissent exactement au bon moment, suivant le rythme de la chanson. Cette amélioration ajouterait une dimension interactive à notre projet, rendant le karaoké plus agréable et facile à suivre pour les chanteurs.

V. Annexe

A. app.py

C'est le premier fichier qui doit être lancé pour pouvoir accéder au site web. Dans le terminal suivant, vous trouverez des liens pour lancer les deux parties de notre site : la page de formulaire et la page de diffusion. Pour la page de formulaire, vous trouverez plusieurs adresses IP, car elle a été configurée en 0.0.0.0. De plus, pour accéder au formulaire depuis n'importe quel appareil, il vous faudra utiliser l'adresse qui n'est pas en 127.0.0.1, car celle-ci est uniquement disponible sur le PC qui l'a lancée.

```
* Serving Flask app '__mp_main__'
* Serving Flask app '__mp_main__'
* Debug mode: off
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://localhost:8000
Press CTRL+C to quit
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:80
* Running on http://192.168.1.10:80
Press CTRL+C to quit
```

B. index.html

Ce fichier html contient le formulaire pour envoyer une chanson vers le fichier *karaoke.html*. Vous devez entrer comme information un nom d'utilisateur pour vous identifier, le titre d'une chanson et son compositeur.

Soumettre une Musique

Nom d'utilisateur

Nom de l'artist

title de la musique

Soumettre

C. index_finish.html

On peut retrouver dans ce fichier la confirmation de l'envoi du formulaire avec les détails de celui-ci.

Détails de la Musique

Nom d'utilisateur : Noah

artist : Luis Fonsi

title : Despacito

C'est votre tour dans 0 chansons

D. karaoke.html

Les paroles et la vidéo se retrouvent ici.

Noah va chanter "Despacito" de Luis Fonsi



Regarder sur YouTube

CHANGER DE CHANSON

Paroles

Oh,
Fonsi!
Dy!
Ooh, oh, no, oh, no, oh
Hey, yeah!
Dididiri Daddy, go!
Yeah, you know that I've been looking at you for a long time
I must dance with you today
Dy!
I saw that the look in your eyes was calling me
Show me the path that I will take
Oh!
You, you're the magnet, and I'm
the metal,
I am getting closer and making a plan
Simply thinking about it makes my heart race
Oh, yeah!
Now, I'm already liking it more than usual
All my senses are asking for more
We cannot do this in a rush
Slowly
I want to breathe in your neck slowly
Let me murmur things in your ear
So you can remember when you're not with me

E. 404.html

Dans ce fichier se trouve une page d'erreur 404 pour les personnes qui n'ont pas rempli le formulaire avant de passer à la page de karaoké ou que la file d'attente est vide.

Erreur 404

Chanson non trouvée

Remplissez le formulaire puis rechargez cette page.

RECHARGER